Neuromorphic
Computing and
Engineering

PURPOSE-LED
PUBLISHING™

PAPER • **OPEN ACCESS**

Neuromorphic control of a simulated 7-DOF arm using Loihi

To cite this article: Travis DeWolf *et al* 2023 *Neuromorph. Comput. Eng.* **3** 014007

View the article online for updates and enhancements.

You may also like

- A spiking central pattern generator for the control of a simulated lamprey robot running on SpiNNaker and Loihi neuromorphic boards
Emmanouil Angelidis, Emanuel Buchholz, Jonathan Arreguit et al.
- Benchmarking spiking neurons for linear quadratic regulator control of multi-linked pole on a cart: from single neuron to ensemble
Shreyan Banerjee, Luna Gava, Aasifa Rounak et al.
- Static hand gesture recognition for American sign language using neuromorphic hardware
Mohammadreza Mohammadi, Peyton Chandarana, James Seekings et al.

This content was downloaded from IP address 38.58.169.211 on 23/06/2026 at 21:10

IOP Publishing Neuromorph. Comput. Eng. 3 (2023) 014007 https://doi.org/10.1088/2634-4386/acb286

NEUROMORPHIC
Computing and Engineering



OPEN ACCESS

RECEIVED
20 September 2022

REVISED
6 December 2022

ACCEPTED FOR PUBLICATION
12 January 2023

PUBLISHED
6 February 2023

Original Content from this work may be used under the terms of the Creative Commons Attribution 4.0 licence.

Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.



PAPER

Neuromorphic control of a simulated 7-DOF arm using Loihi

Travis DeWolf[1,*], **Kinjal Patel**[1], **Pawel Jaworski**[1], **Roxana Leontie**[2], **Joe Hays**[2] and **Chris Eliasmith**[1,3]

[1] Applied Brain Research, Waterloo, Canada
[2] Navy Research Laboratory, Washington, DC, United States of America
[3] University of Waterloo, Waterloo, ON, Canada
[*] Author to whom any correspondence should be addressed.

E-mail: travis.dewolf@appliedbrainresearch.com

Keywords: spiking neural network, neurorobotic, neural arm control, neuromorphic

Abstract

In this paper, we present a fully spiking neural network running on Intel's Loihi chip for operational space control of a simulated 7-DOF arm. Our approach uniquely combines neural engineering and deep learning methods to successfully implement position and orientation control of the end effector. The development process involved four stages: (1) Designing a node-based network architecture implementing an analytical solution; (2) developing rate neuron networks to replace the nodes; (3) retraining the network to handle spiking neurons and temporal dynamics; and finally (4) adapting the network for the specific hardware constraints of the Loihi. We benchmark the controller on a center-out reaching task, using the deviation of the end effector from the ideal trajectory as our evaluation metric. The RMSE of the final neuromorphic controller running on Loihi is only slightly worse than the analytic solution, with 4.13% more deviation from the ideal trajectory, and uses two orders of magnitude less energy per inference than standard hardware solutions. While qualitative discrepancies remain, we find these results support both our approach and the potential of neuromorphic controllers. To the best of our knowledge, this work represents the most advanced neuromorphic implementation of neurorobotics developed to date.

1. Introduction

Neuromorphic hardware has the potential for highly-efficient, low-latency, low-power implementations of spiking neural networks (SNNs). This is of particular interest for end-to-end embedded robotics applications, where sensory processing and control signal calculations are compute intensive, and the latency and power-cost of an implementation directly affects performance.

To be able to take advantage of neuromorphic hardware requires the development of functional SNNs that meet all of the constraints of the target hardware. This poses a challenge, because the development of functional SNNs, even with no hardware constraints to consider, can be very difficult. Most neural network solutions in robotics are developed using rate neurons, and conversion to spiking neurons, when possible, is non-trivial. Previous work [10] has shown how the Nengo neural modeling and simulation package [3] can be used to streamline this process, for building an end-to-end neurorobotic rover controller as well as an adaptive control add-on to a force-based controller. Here, we extend this approach to the significantly more complicated problem of controlling a 7 degree-of-freedom (DOF) arm in a reaching task. We enumerate our stages of development as well as the design heuristics we have used, and closely examine the specifics of adapting a model to run under the constraints of Intel's neuromorphic technology, code named 'Loihi'.

The SNN presented here controls both the position and orientation of the end effector of a 7-DOF arm simulated in Mujoco [23]. The network is an extension of the recurrent error-driven adaptive control hierarchy (REACH) model [11], which models the human motor control system. To successfully implement the highly nonlinear computations necessary for controlling a 7-DOF arm in neurons, we use a hybrid approach to build the network that takes advantage of both deep learning methods and the methods of the neural engineering framework (NEF) [12]. This unique approach allows us to successfully and systematically implement a SNN that runs on neuromorphic hardware and is able to accurately approximate the control of

© 2023 The Author(s). Published by IOP Publishing Ltd

a standard analytical controller. To the best of our knowledge, this work represents the most complex demonstration of a fully spiking neurorobotic controller running on neuromorphic hardware to date.

2. Background

2.1. Related neurorobotic work

Work in neurorobotic tends to focus on sensory processing (e.g. [7, 14, 20]), higher-level aspects of control such as trajectory planning (e.g. [16, 22, 24]), and supplementary adaptive control modules (e.g. [1, 10]). These all represent important aspects of an end-to-end neurorobotic control system, complementary to the work presented here. Examples of controlling complex robotic systems with SNNs using neuromorphic hard are limited, and this area has been identified as an important site of research focus for the field [2, 8]. Below, we review related work that focuses on developing fully SNN for controlling robotic systems.

In [17], the authors developed a low-level spike-based PID controller implemented on an FPGA to directly drives the robotic arm motors using pulse frequency modulation. We view this work as complementary to our own, suggesting an approach for taking the torque control signal, output by our own controller, and directly interfacing with the robot actuators.

In [6] the authors implement an inverse kinematics SNN for an iCub [19] 4-DOF robot arm, trained with a supervised version of spike-timing dependent plasticity. This work applies a neural encoding method where each neuron is sensitive only to a limited range of values of a single joint, such that there is little redundancy in the population representation. The training and simulation time was limiting on the results, and work remains to be done to implement the approach on the full 4-DOF state space and with more complex arms.

The authors of [18] present a neurorobotic controller built using the NEF to perform operational space control for a 3-DOF arm, implemented on the neuromorphic hardware NeuroGrid [4]. Similar to the scaling issues encountered with the REACH model, as described in section 2.2, the increasing complexity of the equations of motion prevent this approach from extending to a 7-DOF arm.

2.2. The REACH model and control equations

The original REACH model controlled a 3-DOF arm operating in the horizontal plane, and consisted of four components: primary motor cortex (M1), cerebellum (CB), premotor cortex, and primary sensory cortex. In this work, we focus on the M1 and CB, responsible for transforming the control signal from operational space to joint space and generating the gravity and inertia compensation terms. We provide the arm feedback and trajectory for the end effector directly to the model, so the primary sensory cortex and the premotor cortex, which process sensory information and generate target trajectories in REACH, are not modeled here.

The basic control signal calculated by the REACH model is an operational space PD signal:

$$\mathbf{u} = k_p \mathbf{J}^{\mathrm{T}} \mathbf{M}_x \mathbf{u}_x + k_v \mathbf{M}\dot{\mathbf{q}} + \mathbf{g}, \tag{1}$$

where $\mathbf{u}$ is the outbound joint torque control signal, $\mathbf{J}$ is the Jacobian matrix that transforms between task space and joint space, $\mathbf{M}$ and $\mathbf{M}_x$ are the joint space and task space inertia matrices, $\mathbf{u}_x$ is the task space control signal specifying the forces to apply to the end effector, $\dot{\mathbf{q}}$ is the vector of joint velocities, $\mathbf{g}$ is the gravity compensation term, and k_p, k_v are the PD gain terms. In the above equation, the first term is calculated by M1, and the rest are calculated by the CB.

In the original REACH model, all connection weights were calculated using the methods of the NEF [12]. This involves sampling the state space, calculating the target function output at each sample point, and performing a matrix inversion to find connection weights to carry out the target function on the neural representation of the input signal. While this approach works well for a 3-DOF planar arm, for the 7-DOF arm the state space is much larger, and the equations of motion are significantly more complicated. As a result, the number of samples required to accurately estimate the target function is exponentially larger, making the matrix inversion intractable. To overcome this, we use a hybrid approach to build the network that takes advantage of deep learning methods to learn approximations of the $\mathbf{J}^{\mathrm{T}}\mathbf{M}_x$, $\mathbf{M}$, and $\mathbf{g}$ terms, and NEF methods to implement the matrix-vector dot products and direct the flow of information.

2.3. Mujoco and ABR control

Mujoco is a highly efficient physics engine designed specifically for robotics simulation that was recently made open-source by DeepMind. The arm model is based on the Kinova Generation 3 7-DOF arm, shown on the right in figure 3. The ABR Control package is a Python library that provides a Python interface for Mujoco, and an API for robotics control. The ABR Control operational space controller is used as the ideal controller for baseline comparison and for the training of the neural controllers.

2.4. Nengo and supporting development packages

The Nengo neural modeling and simulation platform allows users to quickly build models using a Python-based API 'frontend' and to specify different hardware 'backends' for the model to run on. The Nengo backends handle compiling and interfacing with the simulation hardware, and users have the choice of running on CPUs, GPUs, FPGAs, or specialized neuromorphic hardware (such as Intel's Loihi). The same model can be compiled to run on different hardware backends with few to no changes required in the frontend script (although taking advantage of hardware specific features, or avoiding hardware specific constraints, typically requires modifications).

Nengo is a well-tested neural modeling package that has been used in the creation of many different kinds of neural networks, ranging from the world's largest functional brain model [13] to state-of-the-art edge applications [5]. Users are able to draw on Nengo's library of models to quickly build up complex neural networks using pre-built sub-networks, and visualize and interface with their simulation during run-time using the NengoGUI package. We use the NengoDL package [21] to integrate deep networks built in Keras or TensorFlow, and take advantage of the resource distribution manager TensorFlow offers for workload distribution across multiple processors. The NengoInterfaces package offers helper classes to handle the overhead of interfacing with different kinds of sensors and simulators, including connecting, running, communicating, and restarting. The NengoLoihi package provides a backend extension for compiling to and running models on the Loihi, and handles communication with the chip. Additionally, the NengoLoihi package provides a Loihi emulator, simulating the Loihi dynamics and computations, that can be used to test and debug models running locally on a standard workstation. There are many more supporting packages for Nengo, but we limit our review to the above packages relevant to the work presented in this paper.

2.5. The Loihi chip

The target neuromorphic hardware of our neural controller is Intel's Loihi chip [9]. The Loihi chip is a many-core mesh with 128 neuromorphic cores, 3 embedded x86 processing cores, and off-chip communication interfaces. Each neuromorphic core has 1024 neural 'compartments,' where a compartment can be allocated to simulate a neuron or dendrite. An asynchronous network-on-chip communicates packetized messages between cores, allowing write, read request and response, spike messages, and barrier messages for time synchronization to be sent between cores.

The NengoLoihi package handles all of the low-level mapping to and communication with the Loihi. For simple neural models, the details of the Loihi chip are abstracted out and handled automatically when compiling to the chip from Nengo. For more complex models, it is necessary to take into account the specific constraints and features of the Loihi chip in the frontend model description to achieve target performance levels. The constraints addressed as part of this project include: discretized neuron activity profiles, neuron count and incoming/outgoing connectivity limits, connection weight discretization, and on-chip/off-chip communication speeds. Below, we discuss the details of each of these constraints and effects they have on model performance.

2.5.1. Loihi neuron activation profiles

The on-chip discretization of the Loihi means that its neuron activation profiles only roughly approximate standard neural activation. Figure 1 shows the differences in activation profiles for leaky integrate-and-fire and rectified linear neurons on standard hardware and on the Loihi.

For low firing rates, i.e. less than 200 Hz, the activation profiles are relatively similar, but in the higher firing rate ranges the discrepancy becomes significant. As neurons move into these higher firing rates, their representational power becomes limited due to the limited number of possible firing rates. In this way, keeping neuron firing rates in the lower ranges will lead to better representation.

In standard ANNs, neurons are often trained with output in very small ranges, e.g. 0–1. For the method we use here of converting ANNs to SNNs, the goal is for the low-pass filtered spiking output to approximate the rate mode output. Low target firing rates prevent spiking neurons from being able to accurately approximate rate neuron output, as illustrated in figure 2. In this way, higher firing rates will lead to better representations.

These two competing features of SNNs running on the Loihi mean that tuning the spiking neuron firing rates properly is a very important step in converting networks to run on Loihi.

2.5.2. Loihi neuron count and connectivity limits

A single Loihi chip has 128 cores, each with 1,024 compartments, which are allocated to both neuron models and dendrites. Basic neuron models and dendrites each require a single compartment. The neurons and dendrites share a pool of incoming axons. As a result, there is an interplay between the total number of neurons and the incoming/outgoing connectivity of each neuron in determining the size of model that can

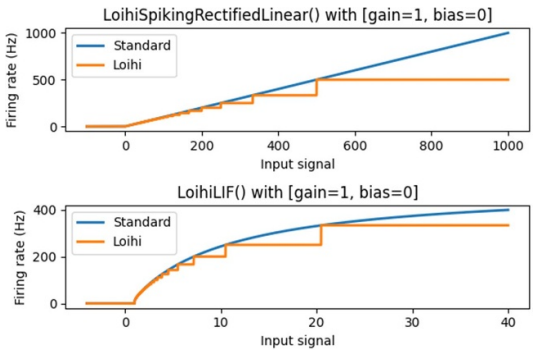


Figure 1. The neural activity profiles of rectified linear (top) and leaky integrate-and-fire (bottom) neurons, in their standard form (blue) and discretized on the Loihi (orange). See text for details.

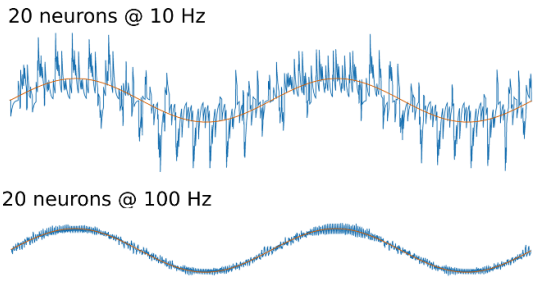


Figure 2. Two populations of neurons representing a sine wave. Neurons with a higher firing rate are able to generate a smoother, more accurate approximation. Top: 20 neurons, max firing rate 10 Hz. Bottom: 20 neurons, max firing rate 100 Hz. See text for details.

fit on a Loihi chip. For models that employ multiple Loihi chips, the arrangement and connectivity of different populations is important.

Inter-chip connectivity and communication on the Loihi is possible, but is highly costly in terms of network latency. The inter-chip routing system on the Loihi has limited resources available for routing spikes in real-time, and quickly builds up a backlog when traffic increases. For real-time applications, such as neurorobotics, multi-chip implementations are only appropriate when there is little to no inter-chip communication.

2.5.3. Loihi weight discretization

Connection weights on Loihi are integer values between −256 and 255 when using 8 bit weights (where weights can take on every other value). To quantize weights to this range, NengoLoihi normalizes each connection using the largest connection weight value as the normalization term. When a connection has multi-dimensional output, and the possible range of values for each output dimensions is significantly different, precision issues can arise. This is because normalizing a connection representing a small magnitude output signal by a much larger value will result in a very course discretization of the connection weights, drastically reducing representational accuracy.

2.5.4. Communication

There are several methods for sending input signals to the Loihi. For our application, low-latency is the primary concern, and the fastest method of sending and receiving a streaming input signal is by sending spikes. The communication latency is directly correlated with the amount of information sent, and the cost of each additional neuron's output is relatively high. As a result, it is critical for real-time applications to minimize the number of neurons sending spikes to/from the chip.

Communication between groups of neurons on-chip also directly affects the speed of simulation. When large numbers of spikes on every time step, the spike routing infrastructure can be overloaded. This overloading results in either spikes being dropped or a slow-down in simulation speed. For real-time applications, both of these results can have significant effects on the system performance.

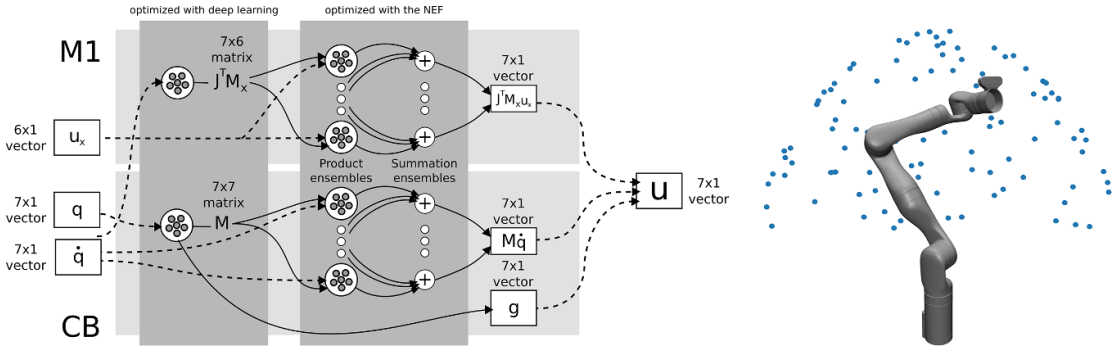


Figure 3. Left: the architecture of our neural controller. The horizontal rectangles encapsulate the M1 (top) and CB (bottom) neural networks. The vertical rectangles encapsulate the subnetworks optimized with deep learning (left) and the neural engineering framework (right). The circular objects represent networks of neurons. Dotted lines represent connections sending information to or from the network. Right: the Mujoco model of the Kinova Generation 3 7-DOF arm, and an example set of targets (blue dots).

3. Methods

3.1. Benchmark task

To benchmark the performance of the neural controller, we developed a reaching task that requires the control of 6 DOF of the end effector in task space: x, y, z, roll, pitch, and yaw. A set of targets are randomly generated on the surface of the upper half of a sphere, with radius 0.75 m, centred at 0.5 m off the ground, as shown on the right in figure 3. The target orientation is calculated to point directly outwards from the first joint of the arm. As the arm moves from target to target, a sweeping arc trajectory is set as the target trajectory, to avoid encountering situations where the arm could fold in on itself and become stuck. While we recognize the importance of avoiding self-collision, we consider it to be outside of the scope of this work. In the remainder of this section we detail each of the four stages we undertook to develop the final model.

3.2. Stage 1: designing a node-based network architecture implementing an analytical solution

The first stage of development is to break down the target algorithm into a circuit-diagram connecting a set of nodes. In this diagram, each node performs a function on its input, and the nodes are connected such that the target algorithm is implemented. There are many different circuit-diagrams that could implement a given algorithm. Keeping in mind that our goal is to replace each node with a set of neurons approximating the same functions, for a time-sensitive application, we adopt the following heuristics. The final network architecture resulting from applying these heuristics is shown in figure 3.

3.2.1. Design heuristic 1

The first heuristic is to separate the calculation of terms combined through linear operations. A group of neurons only needs to be sensitive to multiple input variables if it is calculating a nonlinear function dependent on all variables. For each additional variable a group of neurons is sensitive to, the size of the state space that must be sampled during training increases exponentially. In our application, where the target equation is stated in equation (1), we note that $\mathbf{u}$ is comprised of three independent terms that are summed together.

Theoretically, a network with three nodes (one for $k_p\mathbf{J}^T\mathbf{M}_x\mathbf{u}_x$, one for $k_v\mathbf{M}\dot{\mathbf{q}}$, and one for $\mathbf{g}$) could implement the target equation. The three node network, however, has two nodes with very large input state spaces. The $k_p\mathbf{J}^T\mathbf{M}_x\mathbf{u}_x$ node is dependent on $\mathbf{q}$, to calculate $\mathbf{J}^T\mathbf{M}_x$, and $\mathbf{u}_x$ to calculate the matrix-vector dot product. The $k_v\mathbf{M}\dot{\mathbf{q}}$ node is dependent on $\mathbf{q}$ to calculate $\mathbf{M}$, and $\dot{\mathbf{q}}$ to calculate the matrix-vector dot product. For controlling a 7-DOF arm, these nodes have 13 and 14 dimensional input, which must be very densely sampled due to the high non-linearity of the target functions. In practice, this makes building a sufficiently accurate neural network to approximate this function very difficult.

3.2.2. Design heuristic 2

The second heuristic is to divide nonlinear calculations such that highly nonlinear functions are calculated in neurons sensitive to the minimum number of variables. In some cases, the target function of a node may have one or several sub-terms that are dependent on only a subset of the input variables. In these situations, it can improve accuracy to first compute these sub-terms separately in neural populations sensitive to only

the necessary variables. This decreases the size of the state space that must be sampled when training the neurons to approximate these complex functions, making it easier to achieve high accuracy across all input.

In our network, the $k_p\mathbf{J}^T\mathbf{M}_x\mathbf{u}_x$ term has two sub-terms which are dependent on the same input variables: the $\mathbf{J}^T$ and $\mathbf{M}_x$ sub-terms, which are functions of only $\mathbf{q}$. Because they are both functions of the same variables, we compute them at the same time, and project the output to a network that computes the matrix-vector dot product between $\mathbf{J}^T\mathbf{M}_x$ and $\mathbf{u}_x$.

Here, again, we face the situation of an ensemble with many inputs. The output from $\mathbf{J}^T\mathbf{M}_x$ is 42 dimensional, and $\mathbf{u}_x$ is 6 dimensional, leading to 48 input variables. Fortunately, the dot product calculation is composed of two-element multiplies and a large set of summations, which can be carried out in parallel. The $\mathbf{J}^T\mathbf{M}_x$ and $\mathbf{u}_x$ signals are projected into small neural populations designed to perform two-element multiplies, and then to a layer that sums the resulting products to complete the dot product calculation.

For the $k_v\mathbf{M}\dot{\mathbf{q}}$ term, we also calculate $\mathbf{M}$ separately and send both it and $\dot{\mathbf{q}}$ into a similar matrix-vector dot product network.

3.2.3. Design heuristic 3

The third heuristic is to minimize the number of sequential layers between the input signal and output, to reduce noise and latency. Each additional layer introduces, at a minimum, one additional time step for information to be propagated through the network. When synaptic filters are used to smooth the output of each layer, as is common in SNNs, several effective time steps of delay are introduced. Additionally, representational error is compounded with each successive layer, which can increase the noisiness of the final output. This heuristic serves as a motivation to keep as many calculations combined as possible, somewhat counteracting the effects of the first two heuristics.

The development and design process is highly iterative. As the following stages of implementation are carried out, we often return to this first stage of network architecture design with additional insights on constraints or obstacles to be overcome in the later stages. Figure 3 shows the final network architecture for our neuromorphic controller.

3.3. Stage 2: developing a rate neuron implementation

Given the network architecture from figure 3, we test it to confirm that the generated output exactly matches equation (1). Once we are convinced it is functional, the next step is to develop a rate neuron implementation of that same network. The goal of this stage of development is to replace each of the functional nodes with neural ensembles or networks. In this stage, we begin taking advantage of Nengo's API, that integrates both deep learning methods and NEF methods for learning connection weights, to train the M1, CB, and dot product networks.

3.3.1. The M1 and CB networks

For both the M1 and CB networks, standard deep learning development procedure is followed to find a neural network capable of performing the tasks. Here, the focus is on developing a rate neuron solution, without much consideration for the eventual constraints of spiking neurons or neuromorphic hardware. We find that two layer fully-connected networks, with 20,480 spiking rectified linear neurons per layer, are able to learn the target functions. For the CB network, the gravity term is decoded off of the first layer and the inertia term is decoded off the second layer.

The datasets for training the M1 and CB networks were generated by moving the arm to 50 sets of 100 randomly chosen targets per session, and recording the output from the ideal controller. The networks are trained using rate neurons for 50 epochs with a batch size of 512, mean-squared loss as the objective function, and an initial learning rate of 0.001. The learning rate was reduced every epoch using an exponential decay learning rate scheduler, with decay rate of 0.8. Performance results post-training are shown in figure 4.

3.3.2. Matrix-vector dot product networks

Matrix-vector operations are easy to define in closed form, and thus we use the NEF methods for building neural networks to compute them. Dot product calculations are a series of multiply and summation operations. Similarly, the matrix-vector dot product networks, shown in figure 3, are an arrangement of multiply and summation ensembles. The multiply ensembles have been optimized for multiplying two terms together, under the assumption that the input values are between -1 and 1. For the vector input signals coming from off-chip, i.e. $\mathbf{u}_x$ and $\dot{\mathbf{q}}$, the normalization and mean-subtraction is done before they are sent into the network. For the matrix input signals coming from on-chip, i.e. $\mathbf{J}^T\mathbf{M}_x$ and $\mathbf{M}$, the target output of the M1 and CB networks is normalized and mean-subtracted prior to training. This mean-subtraction and normalization is then undone as part of the target function calculated. The multiply outputs are then summed appropriately to perform the matrix-vector dot product calculation.

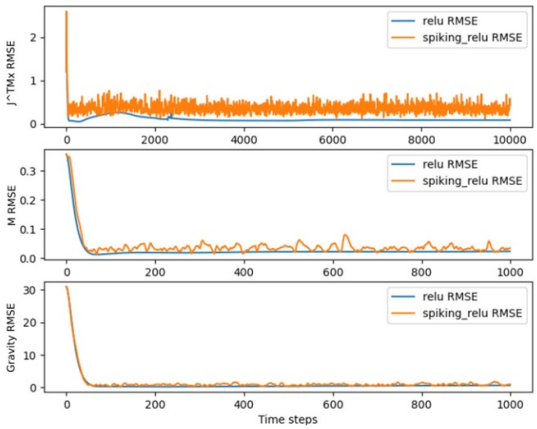


Figure 4. Top: the 2-norm of the trained M1 network measured across the full 42 dimensional output of $\mathbf{J}^{\mathrm{T}}\mathbf{M}_x$, plotted against time as the arm reaches to six targets. Middle: the 2-norm of the trained CB network measured across the full 49 dimensional output of $\mathbf{M}$, plotted against time as the arm reaches to six targets. Bottom: the 2-norm of the trained CB network measured across the full seven dimensional output of $\mathbf{g}$, plotted against time as the arm reaches to six targets. See text for details.

The constant terms for each multiply network were found by running the analytic controller, recording input to the dot product operation, and finding the mean and standard deviation of the input. Three times the standard deviation was used as the scaling term, to ensure that 98% of the input will be inside the -1 to 1 range. Figure 5 shows the results from an example dot product network running with spiking neurons.

3.4. Stage 3: retraining the network to account for spiking neurons and temporal dynamics

The methods used to build each of the networks, i.e. deep learning or the NEF, determines what kind of process is necessary to convert each to running with spiking neurons.

3.4.1. The M1 and CB networks

For networks built using deep learning, we need to account for noise introduced by running inference with spiking neurons, to tune the firing rates of neurons to be in a range appropriate for spiking neurons, and to find appropriate synaptic filter time constants for each of the connections. To make the network robust to spiking noise, we train the network with noise in the neural activation profiles or added in between layers. There are several methods for bringing the neuron firing rates into a functional range. The most common way is to add a firing rate regularization term to the loss function. Another way of bringing the firing rates into a desired range, which we use here, is to scale down the amplitude of the output from each neuron. This forces the firing rates of each of the neurons to be driven up to achieve the same target output signal, which leads to convergence during training in a range of firing rates more amenable to spiking neurons.

Determining appropriate time constants on the synaptic filters is based on examining the trade-off between noise in the output and propagation delay in the network. Typically, we set the synaptic filters to zero, examine system performance on the task, and then look at recorded data from the output of each network or network layers to find points where significant noise is introduced. There is an interplay between the firing rates and synaptic filter choice, as highlighted in figure 2. Neurons with higher firing rates require less filtering to smooth the output, where neurons with lower firing rates require larger filter time constants. This part of network tuning can be automated with hyperparameter search software, but it can often be faster to manually tune the synaptic filter time constants.

Figure 4 shows the performance of the networks with rate neurons (blue) and spiking neurons (orange). The top plot shows the M1 network, the lower two plots show the CB network, which learns both the gravity term and inertia matrix.

3.4.2. Matrix-vector dot product networks

For networks built with the NEF, Nengo automatically takes into account spiking noise when generating connection weights, and sets appropriate default firing rates and parameters for the neurons. The focus during conversion for these networks is largely on finding appropriate synaptic filter time constants, using the same approach as described above.

Figure 5 shows the performance of an example dot product network running with both rate and spiking neurons compared to an analytical calculation.

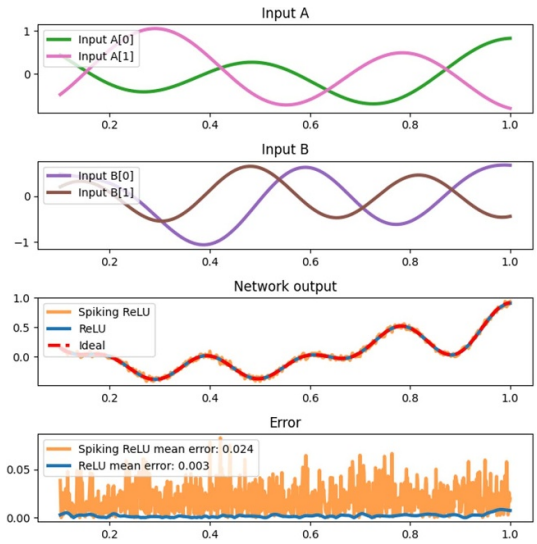


Figure 5. Ground truth output is dotted red. Input A: the randomly generated two-dimensional input signal representing the (1, 2) input vector. Input B: the randomly generated two-dimensional input signal representing the (2, 1) input vector. Network output: the dot product network output running in analytical mode (dotted red line), with rate neurons (blue line), and with spiking neurons (orange line). Error: the error over time of the rate neurons (blue line) and spiking neurons (orange line) network implementations.

3.5. Stage 4: adapting the network for Loihi

The last stage of development for our neurorobotic controller is taking into account the constraints of the target neuromorphic hardware, the Loihi chip.

3.5.1. Loihi neuron activation profiles

We address the unique shape of the discretized neuron activation profiles on the Loihi by using rate-mode versions of these neurons for evaluation during training, provided by NengoLoihi. This allows the network to account for the quantization effects during training, which greatly improves the performance of the networks running on-chip. The firing rates for each layer of the network were tuned to be on average between 100–150 Hz, achieved by scaling the amplitude of each neuron down by 100 during training.

3.5.2. Neuron count and connectivity limitations

The M1 and CB implementations developed in the previous stages were two layer fully-connected networks with 20,480 rectified linear neurons per layer. To implement these on Loihi, the number of compartments required, assuming 1 compartment per neuron and 1 per synapse, would be at least $20{,}480 + 20{,}480 = 40{,}960$, and the amount of memory would be $20{,}480^2 = 419{,}430{,}400$ bytes. Where each chip has 128 cores with 1,024 compartments and 128 KB synapse memory per core, and the maximum number of chips on a board is 32. Thus each chip has about 16 MB and each board about 512 MB. Consequently, just one such network would require the full memory of the largest board.

To address this issue, two changes were made to the network designs: the networks were rearranged to be deeper with fewer neurons per layer; and we reduced the number of neurons per layer as much as possible, while maintaining performance. The depth of the network was kept to the minimum able to perform the task accurately, in accordance with the third design heuristic, to minimize the time required for information to propagate through the network.

The final M1 network used was a four layer fully-connected neural network with 224 neurons in the first layer and 512 neurons per layer in the remaining three layers. The final CB network was implemented as a four layer fully-connected, multi-headed neural network, with 224 neurons in the first layer and 512 neurons per layer in the remaining three layers. The gravity compensation and the joint space inertia matrix terms were both decoded off the 4th layer.

3.5.3. Weight discretization

The NengoLoihi emulator allows for simulation with and without weight discretization. As expected, weight quantization generally degrades the performance of the network.

In particular, the M1 dot product performance significantly degraded with the discretized weights. Upon investigation, we found that two dimensions of the seven-dimensional output signal represented values 10–100× smaller than the other dimensions. As described in section 2.5.3, this issue leads to errors on the

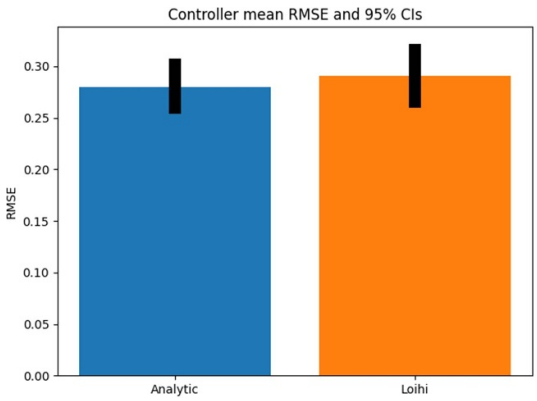


Figure 6. Mean RMSE and 95% confidence intervals of the analytic controller (orange bar) and spiking controller without adaptive control (blue bar) across ten trials of the benchmark task. See text for details.

Loihi. To address this, we normalized the signal of each dimension, which removed the large discrepancies between connection weight magnitudes, and scaled them back to original ranges on a downstream connection. The scaling values were found by analyzing the ranges of the output signal of the M1 dot product network when running the ideal controller, and calculating values to roughly normalize the ranges of each dimension.

3.5.4. Communication

As described in section 2.5.4, the fastest way of sending streaming input to the Loihi is by sending spikes. For deep learning trained networks, a small layer of neurons that runs off-chip was added to the frontend of each network, whose primary role is to efficiently convert the input signal to spikes for the rest of the network. For output from deep learning trained networks, small final output layers are common practice already. This is ideal for our application here, as it minimizes the number of neurons that must be probed to transfer the signal off-chip.

For NEF networks, NengoLoihi provides a small, optimized layer of neurons for efficiently represent signals, called 'DecodeNeurons'. These are automatically added to the network during compilation on connections to or from NEF ensembles to reduce communication bandwidth in the network, for both on-chip/off-chip communication and communication between two NEF ensembles on-chip. In a basic DecodeNeuron layer, each dimension is represented by ten neurons. Five neurons are tuned to negative values and five neurons are tuned to positive values. Using the NEF methods to weighting the output firing rates from all ten neurons at each point in time, the value of the input signal can be estimated. For an NEF connection from an ensemble of n neurons representing d dimensions to a population of m neurons representing d dimensions, DecodeNeurons result in $n(10d) + (10d)m$ connection weights instead of nm connection weights. Because $10\,d$ is typically significantly less than n or m, the use of intermediary DecodeNeurons ensembles offers a significant reduction in required bandwidth.

4. Results

Here we present compare results from the ideal, analytical controller and the final spiking controller running on the Loihi. To ensure the performance across the workspace is well evaluated, we test 10 sets of 5 targets. Each test runs for 30 simulated seconds, with the target location and orientation changing every 5 seconds.

Figure 6 shows the mean RMSE and 95% confidence intervals from the analytical, rate neuron, spiking neuron, and Loihi controllers. The analytic controller has a mean of 0.27951, and the Loihi controller has a mean of 0.29106, which is an increase in error of 4.13%.

To provide a better sense of the movements of the controller, we also present the task-space errors and end effector trajectories on a longer, 50 second, representative trial. Figure 7 top shows the distance to the target for each of the six dimensions under control in task space plotted against time: the (x, y, z) position as well as (alpha, beta, gamma) Euler angles of orientation. The last row is a plot of the 2-norm of the distance to target across all six dimensions over time. Note that the target changes every 5 seconds, as reflected in the sharp increases in error at those times. The mean RMSE on this trial for the analytic controller is 0.3184, and the mean RMSE for the Loihi controller is 0.3264.

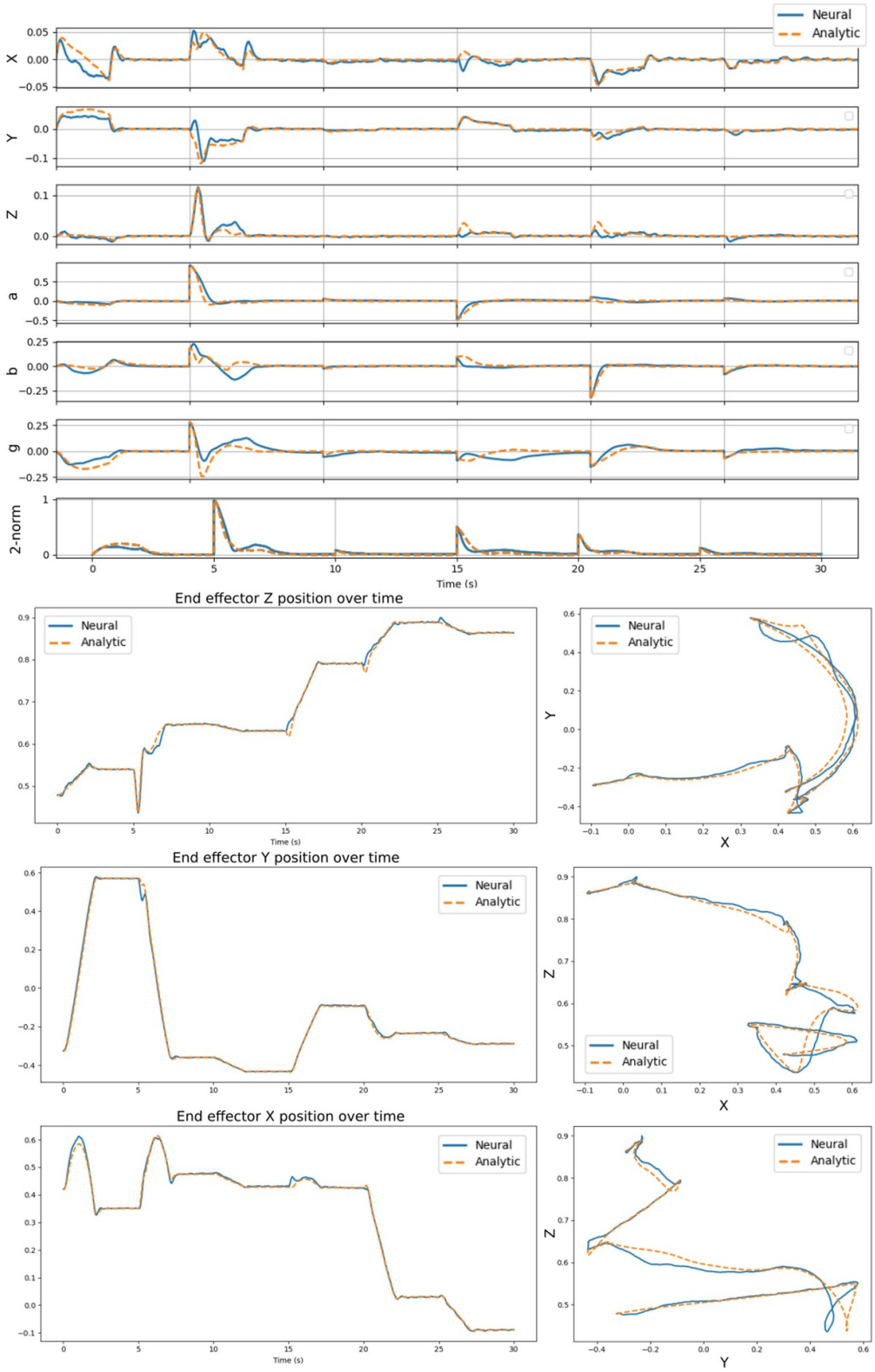


Figure 7. Benchmark task results of the analytic controller (dotted orange line) and SNN controller running on Loihi (blue line). Top: the error along each dimension of the task space, as well as the 2-norm of the task space error. Bottom left: the X (top), Y (middle), and Z (bottom) trajectories of the end effector. Bottom right: plots of the (x, y), (x, z), and (y, z) trajectories of the end effector. See text for details.

The lower plots in figure 7 shows the X, Y, and Z trajectories of the controllers on the same representative trial. On the left the X, Y, and Z trajectories are plotted against time. On the right, the (X, Y), (X, Z), and (Y, Z) trajectories are plotted, showing the 3D movement of the arm as it reaches to each target.

Figure 8 shows the power and latency benchmarks from running the neural controller on a high-end CPU, GPU, and the Loihi neuromorphic hardware. We also compare performance against a standard

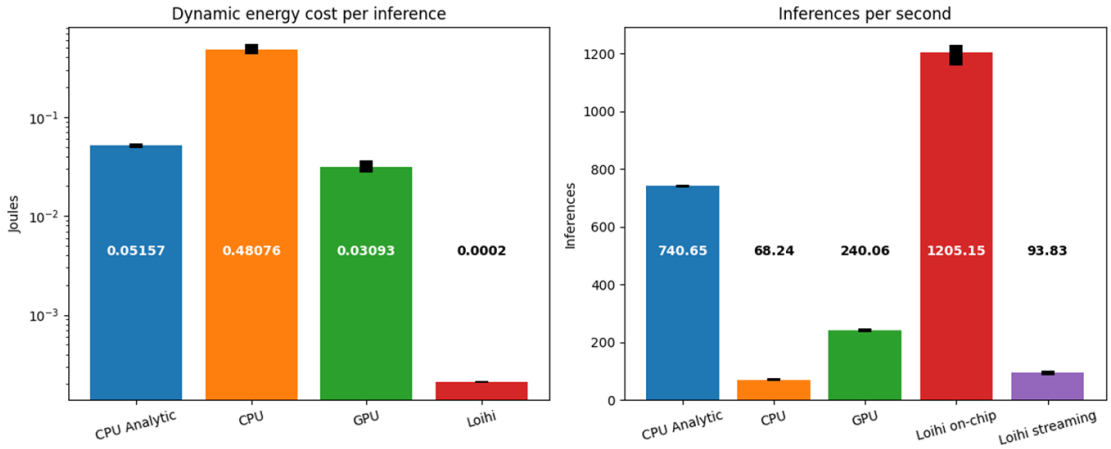


Figure 8. A comparison of the energy costs and throughput of the network implemented on standard CPU, GPU, and the Loihi hardware, compared with an algorithmic implementation written in Python (labeled 'CPU Analytic'). Left: the dynamic energy cost of running each network. The cost of running the network on Loihi is two orders of magnitude less than the nearest alternative implementation. Right: the number of inferences per second of each implementation. 'Loihi on-chip' refers to the data being pre-loaded on the board, where 'Loihi streaming' refers to sending input and output to and from the Loihi board at every time step (necessary for real-time closed loop applications). See text for details.

algorithmic implementation of the controller running on a CPU. The 'dynamic energy' cost is presented, which removes the background energy cost of running the hardware. Throughput information is collected for the Loihi with both the data pre-loaded on the board (i.e. running open-loop), and with streaming input to and from the board (i.e. running closed-loop). For collecting the power and latency results, the Intel Neuromorphic Research Cloud was used to access the Nahuku 32-chip board for Loihi results. A workstation with an AMD Ryzen 9 5950x @ 3.75 GHz CPU, with 128 GB DDR4 @ 3600 Hz RAM, and a RTX 3090 GPU was used to collect the CPU and GPU results.

5. Discussion

In this paper, we presented a neurorobotic controller running on Loihi neuromorphic chip capable of controlling the position and orientation of the end effector of a 7-DOF arm in simulation. A four stage approach was applied to develop this controller, starting from a target algorithm and working down to a detailed neuromorphic implementation. Our final neurorobotic controller achieves a mean RMSE of 0.291, only 4.13% higher than the analytic controller mean value 0.2795.

The power cost and latency results presented show that the neuromorphic implementation on Loihi reduces the dynamic energy cost per inference by two orders of magnitude. Low latency is often considered another primary advantage of neuromorphic implementations. The Loihi chip provides very fast inference when the data is pre-loaded and can be fed into the network as soon as the network is ready, and outperforms CPUs and GPUs in all cases. However, the current on-chip/off-chip communication in Loihi introduces significant delay, reducing the number of inferences per second from 1,205.15 to 93.83. This is better than the CPU SNN implementation, but slower than the GPU implementation and analytic CPU implementation. Notably, we are using the first generation Loihi hardware in this work, and the recently released second generation chip is reported to have addressed this issue [15].

In the original REACH model, a nonlinear adaptive component was included, able to account for unmodeled dynamics and external perturbing forces affecting movement. In [10] we showed how this nonlinear adaptive control can be implemented on the Loihi. Here, the adaptive control was successfully implemented at all stages of development up to the point of weight quantization for running on the Loihi. When running on the Loihi, the adaptation provided only a minor improvement if any, and performance varied unreliably with different trajectories. The current software infrastructure does not allow for using an on-chip signal as a training signal for another on-chip ensemble. As a result, the training signal for adaptive control is calculated on-chip, sent off-chip, and then sent back on-chip for learning. At each step of communication, both noise and latency are added to the signal, and we believe this is why we do not see the expected performance gains with our adaptive control implementation.

We present this work as a proof-of-concept of fully spiking neurorobotic solutions. Effective control is a critical part of end-to-end neuromorphic systems, and we believe that this work demonstrates both a promising step and successful approach for building neuromorphic controllers. The functional separation of

different components of the network allows us to use whichever connection weight optimization method is most advantageous for each component. The NEF allows us to build prior knowledge into the network, such as in the dot product calculations, and deep learning allowed us to learn the highly nonlinear Jacobian and inertia matrix functions. The deep learning was implemented here with an exact reference implementation that could be used to set up a supervised learning scenario. It is foreseeable that other approaches, such as reinforcement learning, could also be used to train up these specific sub-networks for systems without exact reference implementations.

Future work lies in connecting this controller to a physical robot arm, and further optimizing the controller to reduce latency and improve our performance metrics in a real-world setting. We are beginning to undertake this work, with a long term goal of implementing a fully adaptive, SNN controller running on neuromorphic hardware performing useful real-world tasks.

Data availability statement

The data that support the findings of this study are available upon reasonable request from the authors.

Acknowledgments

C E was supported by CFI (52479-10006) and OIT (35768) infrastructure funding as well as the Canada Research Chairs program and AFOSR Grant FA9550-17-1-0026.

Author contributions

T D, J H, and C E conceived of and designed the project. T D, K P, P J, and R L developed and tested the code, J H and C E consulted throughout development. T D wrote the first draft of the paper. C E, J H, and R L edited and revised.

ORCID iDs

Travis DeWolf https://orcid.org/0000-0002-8780-5631
Joe Hays https://orcid.org/0000-0001-5922-703X

References

[1] Antonietti A, Martina D, Casellato C, D'Angelo E and Pedrocchi A 2019 Control of a humanoid NAO robot by an adaptive bioinspired cerebellar module in 3D motion tasks *Comput. Intell. Neurosci.* **2019** 4862157
[2] Bartolozzi C, Indiveri G and Donati E 2022 Embodied neuromorphic intelligence *Nat. Commun.* **13** 1–14
[3] Bekolay T, Bergstra J, Hunsberger E, DeWolf T, Stewart T C, Rasmussen D, Choo X, Voelker A R and Eliasmith C 2014 Nengo: a Python tool for building large-scale functional brain models *Front. Neuroinform.* **7** 48
[4] Benjamin B V, Gao P, McQuinn E, Choudhary S, Chandrasekaran A R, Bussat J-M, Alvarez-Icaza R, Arthur J V, Merolla P A and Boahen K 2014 Neurogrid: a mixed-analog-digital multichip system for large-scale neural simulations *Proc. IEEE* **102** 699–716
[5] Blouw P, Choo X, Hunsberger E and Eliasmith C 2019 Benchmarking keyword spotting efficiency on neuromorphic hardware *Proc. 7th Annual Neuro-Inspired Computational Elements Workshop* pp 1–8
[6] Bouganis A and Shanahan M 2010 Training a spiking neural network to control a 4-DoF robotic arm based on spike timing-dependent plasticity *2010 Int. Joint Conf. on Neural Networks (IJCNN)* (IEEE) pp 1–8
[7] Bucci L D, Chou T-S and Krichmar J L 2014 Sensory decoding in a tactile, interactive neurorobot *2014 IEEE Int. Conf. on Robotics and Automation (ICRA)* (IEEE) pp 1909–14
[8] Christensen D V *et al* 2022 2022 roadmap on neuromorphic computing and engineering *Neuromorph. Comput. Eng.* **2** 022501
[9] Davies M *et al* 2018 Loihi: a neuromorphic manycore processor with on-chip learning *IEEE Micro* **38** 82–99
[10] DeWolf T, Jaworski P and Eliasmith C 2020 Nengo and low-power AI hardware for robust, embedded neurorobotics *Front. Neurorobot.* **14** 568359
[11] DeWolf T, Stewart T C, Slotine J-J and Eliasmith C 2016 A spiking neural model of adaptive arm control *Proc. R. Soc.* B **283** 20162134
[12] Eliasmith C and Anderson C H 2003 *Neural Engineering: Computation, Representation and Dynamics in Neurobiological Systems* (Cambridge, MA: MIT Press)
[13] Eliasmith C, Stewart T C, Choo X, Bekolay T, DeWolf T, Tang Y and Rasmussen D 2012 A large-scale model of the functioning brain *Science* **338** 1202–5
[14] Friedl K E, Voelker A R, Peer A and Eliasmith C 2016 Human-inspired neurorobotic system for classifying surface textures by touch *IEEE Robot. Autom. Lett.* **1** 516–23
[15] Intel 2022 *Intel Advances Neuromorphic with Loihi 2, New Lava Software Framework and New Partners* (available at: www.intel.com/content/www/us/en/newsroom/news/intel-unveils-neuromorphic-loihi-2-lava-software.html)
[16] Lele A, Fang Y, Ting J and Raychowdhury A 2021 An end-to-end spiking neural network platform for edge robotics: from event-cameras to central pattern generation *IEEE Trans. Cogn. Dev. Syst.* **14** 1092–103
[17] Linares-Barranco A, Perez-Peña F, Jimenez-Fernandez A and Chicca E 2020 ED-BioRob: a neuromorphic robotic arm with FPGA-based infrastructure for bio-inspired spiking motor controllers *Front. Neurorobot.* **14** 590163

[18] Menon S, Fok S, Neckar A, Khatib O and Boahen K 2014 Controlling articulated robots in task-space with spiking silicon neurons *5th IEEE RAS/EMBS Int. Conf. on Biomedical Robotics and Biomechatronics* (IEEE) pp 181–6
[19] Metta G, Sandini G, Vernon D, Natale L and Nori F 2008 The iCub humanoid robot: an open platform for research in embodied cognition *Proc. 8th Workshop on Performance Metrics for Intelligent Systems* pp 50–56
[20] Pearson M J, Pipe A G, Melhuish C, Mitchinson B and Prescott T J 2007 Whiskerbot: a robotic active touch system modeled on the rat whisker sensory system *Adapt. Behav.* **15** 223–40
[21] Rasmussen D 2019 NengoDL: combining deep learning and neuromorphic modelling methods *Neuroinformatics* **17** 611–28
[22] Siebert L, Cottrell E, Venzke E and Andrews B 2015 Earth's volcanoes and their eruptions: an overview *The Encyclopedia of Volcanoes* ed H Sigurdsson, B Houghton, S McNutt, H Rymer and J Stix (Amsterdam: Elsevier) pp 239–55
[23] Todorov E, Erez T and Tassa Y 2012 Mujoco: a physics engine for model-based control *2012 IEEE/RSJ Int. Conf. on Intelligent Robots and Systems* (IEEE) pp 5026–33
[24] Wei H, Bu Y and Zhu Z 2020 Robotic arm controlling based on a spiking neural circuit and synaptic plasticity *Biomed. Signal Process. Control* **55** 101640